

POO STORE

1 - Apresentação do sistema

1.1 que é o Sistema

O POO Store é um sistema de loja virtual desenvolvido com base nos quatro pilares da Programação Orientada a Objetos (POO): encapsulamento, herança, polimorfismo e abstração. O sistema simula o funcionamento de um, permitindo o cadastro de produtos, gestão de clientes, realização de pedidos e controle de estoque, tudo estruturado por meio de classes e objetos bem definidos.

1.2 Para Quem Serve

O sistema é voltado para pequenos e médios lojistas que desejam disponibilizar seus produtos online. A arquitetura orientada a objetos garante que o código seja organizado, reutilizável e de fácil manutenção.

1.3 Principais Funcionalidades

O sistema contempla as seguintes funcionalidades principais:

- Cadastro de Produtos: registro de itens com nome, descrição, preço e quantidade em estoque, utilizando classes especializadas por categoria .: Eletrônico, Vestuário, Alimento).
- Gestão de Clientes: cadastro de clientes com dados pessoais e histórico de compras, com distinção entre Cliente Comum e Cliente Premium.
- Carrinho de Compras: adição e remoção de produtos, com cálculo automático do total e aplicação de descontos conforme o tipo de cliente.
- Realização de Pedidos: geração de pedidos com status atualizável (Pendente, Em processamento, Enviado, Entregue), encapsulando as regras de negócio.
- Controle de Estoque: atualização automática da quantidade disponível ao confirmar um pedido, com alertas de estoque baixo.
- Formas de Pagamento: suporte a múltiplos métodos (Cartão de Crédito, Boleto, Pix), cada um implementado como uma classe concreta derivada de uma interface abstrata Pagamento.

2 - Modelagem do sistema

2.1 Classes

Para o sistema de loja virtual, foram pensadas classes simples que representam as principais partes do funcionamento da aplicação. Como a loja vende produtos variados, as classes foram organizadas para permitir o cadastro de produtos, login de clientes e administradores, montagem do carrinho, finalização de pedidos, simulação de pagamento e registro de endereço de entrega.

Classe	Responsabilidade
Usuario	Classe abstrata base. Armazena nome, email e senha. Define os métodos Login() e AcessarSistema() como abstratos, obrigando cada subclasse a implementar seu próprio comportamento.
Cliente	Classe abstrata que herda de Usuario. Representa o consumidor final. Possui Carrinho e Endereco. Define o método abstrato AplicarDesconto().
ClienteComum	Herda de Cliente. Implementa AplicarDesconto() retornando o total sem alteração — sem desconto.
ClientePremium	Herda de Cliente. Implementa AplicarDesconto() aplicando 10% de desconto no total da compra.
Administrador	Herda de Usuario. Gerencia produtos, estoque e pedidos. Implementa AcessarSistema() com comportamento próprio de administrador.
Produto	Representa os itens vendidos. O atributo Estoque usa private set — só pode ser alterado pelos métodos AdicionarEstoque() e RemoverEstoque().
Categoria	Organiza os produtos por tipo. Possui nome e descrição.
Carrinho	Armazena temporariamente os produtos escolhidos pelo cliente. Possui métodos para adicionar item, remover item, calcular total e limpar o carrinho.
ItemCarrinho	Representa um produto dentro do carrinho junto com a quantidade. Calcula o próprio subtotal pelo método Subtotal().
Pedido	Representa a compra finalizada. Possui lista de itens, valor total, data, status (enum), tipo de pagamento e endereço. Possui os métodos AtualizarStatus() e CalcularTotal().
Pagamento	Classe abstrata. Define os atributos Id, Forma, Valor e Status, e o método abstrato ProcessarPagamento().
PagamentoPix	Herda de Pagamento. ProcessarPagamento() exibe a chave Pix e confirma o pagamento.
PagamentoCartao	Herda de Pagamento. ProcessarPagamento() calcula as parcelas e aprova o cartão.
PagamentoBoleto	Herda de Pagamento. ProcessarPagamento() gera a data de vencimento e exibe o boleto.
Endereco	Guarda os dados de entrega: rua, número, bairro, cidade, estado e CEP.

2.2 Relação entre as classes

O sistema terá uma classe `Usuario`, que servirá como base para `Cliente` e `Administrador`, pois ambos possuem dados em comum, como nome, email e senha.

O `Cliente` poderá possuir um `Carrinho`, onde serão adicionados os produtos escolhidos. O `Carrinho` será formado por vários `ItemCarrinho`, e cada `ItemCarrinho` estará relacionado a um `Produto`.

Cada `Produto` poderá pertencer a uma `Categoria`, permitindo organizar melhor os itens da loja.

Quando o cliente finalizar a compra, o carrinho dará origem a um `Pedido`. Esse pedido terá uma lista de itens, um valor total, um status e estará relacionado a um pagamento simulado. Além disso, o pedido utilizará um `Endereco` para representar o local de entrega.

O `Administrador` será responsável por gerenciar os produtos, categorias, estoque e acompanhar os pedidos realizados pelos clientes.

2.3 Explicação do Diagrama de Classes — Loja Virtual

2.4 Estrutura de Usuários (Herança)

A autenticação e o controle de acessos da plataforma utilizam o conceito de herança para reaproveitamento de código e definição de papéis claros:

Usuario (Classe Base)

Concentra as credenciais essenciais comuns a qualquer pessoa que acesse o sistema: nome, email e senha , além do método principal de autenticação `login()` .

Cliente (Especialização)

Herda de `Usuario` . Representa o consumidor final. Possui associações diretas como `Endereço` de entrega e com o seu `Carrinho` de compras ativo. Suas ações básicas envolvem navegar no catálogo e acessar o sistema.

Administrador (Especialização)

Herda de `Usuario` . Representa a equipe de gestão da loja. Não realiza compras, mas possui permissões para gerenciar o catálogo de produtos e acompanhar o andamento dos pedidos gerais.

2.5 Catálogo de Produtos

A organização e a oferta de mercadorias no sistema ocorrem por meio de duas entidades ligadas por associação

simples:

Produto e Categoria

A classe Produto gerencia o preço, as informações básicas e o inventário (com métodos para acrescentar ou deduzir do estoque físico). Todo produto está associado de forma obrigatória a Categoria (ex: "Eletrônicos", "Moda"), o que viabiliza a organização interna do catálogo.

2.6 Fluxo de Compra e Pedidos

O ciclo que transforma a intenção de compra do cliente em um faturamento real é estruturado por cinco classes dinâmicas:

Carrinho e ItemCarrinho

O Carrinho é um container temporário do Cliente. Para permitir que o mesmo produto seja comprado em quantidades variadas, o carrinho armazena uma lista de ItemCarrinho. Cada item faz o vínculo entre o Produto específico e a quantidade selecionada, calculando o seu próprio subtotal.

Pedido e Pagamento

Ao fechar a compra, o carrinho é convertido em um Pedido definitivo. O pedido registrado é imutável daquele momento (data da compra, valor total fixado e status da entrega) e herda endereço de destino. Há uma relação de composição estrita com a classe Pagamento, encarregada de processar financeiramente a transação.

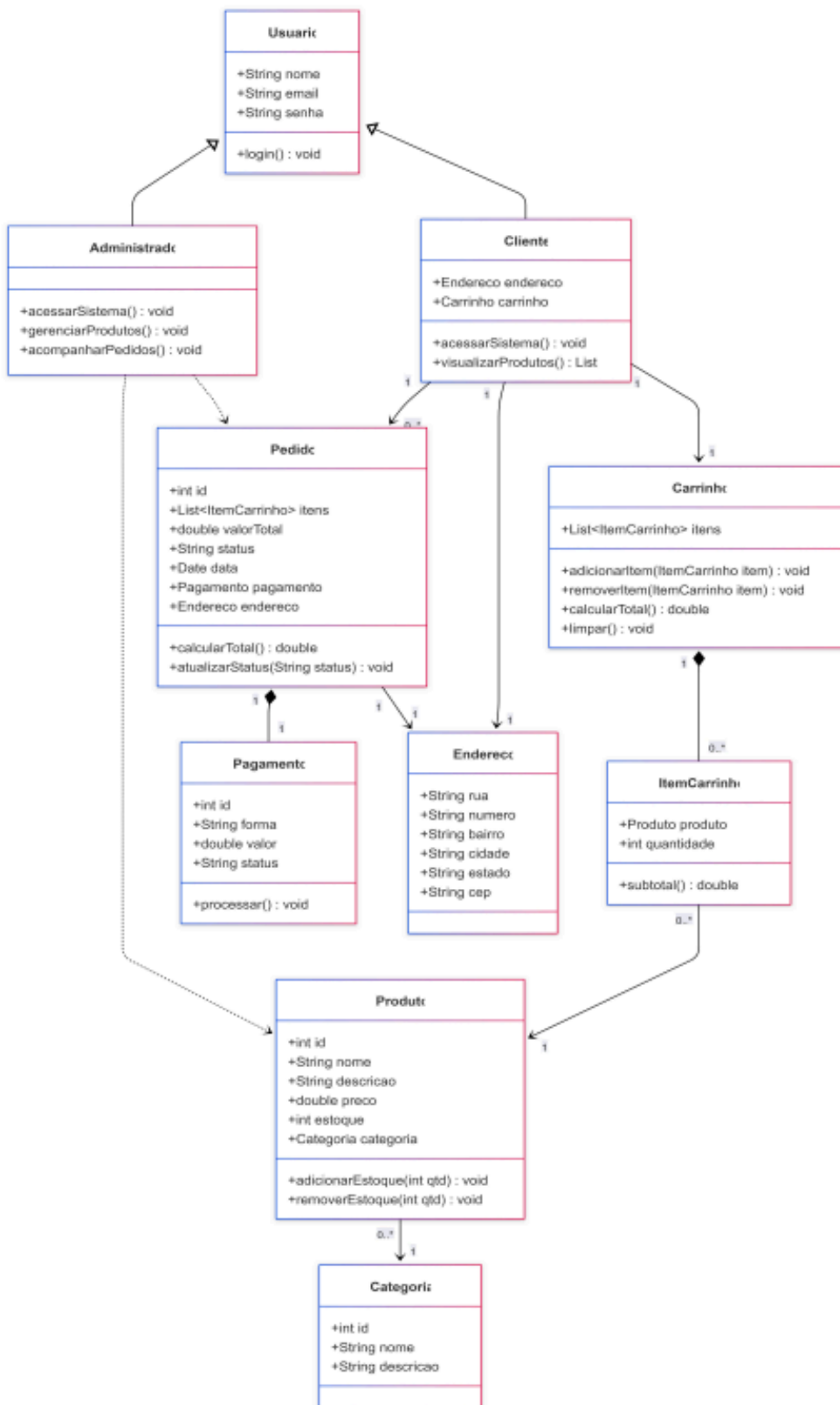
2.7 Resumo Teórico dos Relacionamentos

- **Herança (|):** e estendem a classe.

<|-- Cliente Administrador

Usuario

- **Composição (*--):** Relação de forte dependência existencial. Se um Carrinho ou um Pedido excluído do banco de dados, seus respectivos itens e dados deixam de existir imediatamente.
- **Associação (-->):** Conexões direcionadas de navegação (ex: o item sabe qual produto referência, e o pedido sabe qual endereço deve utilizar para a entrega).
- **Dependência (..):** O não armazena produtos ou pedidos de forma fixa em seus atributos, mas depende dessas classes pontualmente ao executar suas rotinas de gerenciamento.



3 - Aplicação dos quatro pilares da POO

3.1 Abstração

A abstração consiste em representar apenas as informações relevantes de um objeto, sem expor detalhes desnecessários. No POO Store, duas classes abstratas aplicam esse conceito diretamente.

A classe Usuario é abstrata porque não existe "usuário genérico" no sistema — sempre é um Cliente ou um Administrador. Ela define apenas o que é comum aos dois: nome, email, senha e o método Login(). O método AcessarSistema() é declarado como abstrato, obrigando cada subclasse a definir seu próprio

comportamento.

```
public abstract class Usuario
{
    public string? Nome { get; set; }
    public string? Email { get; set; }
    public string? Senha { get; set; }

    public bool Login(string email, string senha)
        => Email == email && Senha == senha;

    // Abstrato: cada subclasse implementa do seu jeito
    public abstract void AcessarSistema();
}
```

O mesmo vale para Pagamento: não existe "pagamento genérico", sempre é Pix, Cartão ou Boleto. A classe define os atributos comuns (Valor, Forma, Status) e declara ProcessarPagamento() como abstrato.

```
public abstract class Pagamento
{
    public decimal Valor { get; set; }
    public string? Forma { get; set; }
    public string Status { get; set; } = "Pendente";

    public abstract void ProcessarPagamento();
}
```

3.2 Encapsulamento

O encapsulamento protege os dados de uma classe, permitindo que sejam alterados apenas por métodos controlados. Na classe Produto, o atributo Estoque usa private set, o que impede que qualquer parte do sistema altere o valor diretamente (como produto.Estoque = 999). A única forma de modificar o estoque é pelos métodos da própria classe, que também garantem a regra de negócio: o estoque não pode ficar negativo.

```
public class Produto
{
```

```

    public decimal Preco { get; set; }

    // private set: ninguém fora da classe pode alterar diretamente
    public int Estoque { get; private set; }

    public void AdicionarEstoque(int quantidade)
    {
        Estoque += quantidade;
    }

    public bool RemoverEstoque(int quantidade)
    {
        // Regra de negócio: não deixa o estoque ficar negativo
        if (quantidade > Estoque) return false;

        Estoque -= quantidade;

        return true;
    }
}

```

3.3 Herança

A herança permite que uma classe reutilize atributos e métodos de outra, evitando repetição de código. O projeto possui duas hierarquias de herança.

Na hierarquia de usuários, Cliente e Administrador herdam de Usuario, recebendo automaticamente nome, email, senha e o método Login(). ClienteComum e ClientePremium herdam de Cliente, recebendo também Carrinho e Endereco.

Usuario (abstrata)

├── Cliente (abstrata)

| ├── ClienteComum

| └── ClientePremium

└─ Administrador

Na hierarquia de pagamentos, as três formas herdam de Pagamento e recebem os atributos Valor, Forma e Status sem precisar repeti-los.

Pagamento (abstrata)

└─ PagamentoPix

└─ PagamentoCartao

└─ PagamentoBoleto

3.4 Polimorfismo

O polimorfismo permite que o mesmo método produza comportamentos diferentes dependendo do tipo do objeto em execução.

Exemplo 1 — AcessarSistema() e AplicarDesconto():

O método AcessarSistema() é herdado de Usuario, mas Cliente e Administrador o implementam de formas diferentes. O mesmo vale para AplicarDesconto(): ClienteComum devolve o total intacto; ClientePremium aplica 10% de desconto.

```
// Em Cliente:
```

```
public override void AcessarSistema()

    => Console.WriteLine($"{Nome} entrou no sistema como cliente.");
```

```
// Em Administrador:
```

```
public override void AcessarSistema()

    => Console.WriteLine($"{Nome} entrou no sistema como administrador.");
```

```
// Em ClienteComum:
```

```
public override decimal AplicarDesconto(decimal total) => total;
```

```
// Em ClientePremium:
```

```
public override decimal AplicarDesconto(decimal total)

{

    decimal desconto = total * 0.10m;

    return total - desconto;

}
```

Exemplo 2 — ProcessarPagamento():

A mesma chamada `pagamento.ProcessarPagamento()` produz saídas completamente diferentes dependendo do tipo de pagamento instanciado. O sistema não precisa verificar o tipo com `if/else` — o C# resolve automaticamente qual implementação executar.

```
// PagamentoPix: exhibe chave Pix e confirma  
  
// PagamentoCartao: calcula parcelas e aprova o cartão  
  
// PagamentoBoleto: gera data de vencimento e exhibe o boleto  
  
pagamento.ProcessarPagamento();           // mesmo comando,  
comportamentos diferentes
```

4 - Decisões Iniciais

4.1 Formato de Armazenamento

Após análise das opções disponíveis, o grupo optou pelo formato JSON para o armazenamento dos dados do sistema. A escolha se justifica pelos seguintes motivos:

Legibilidade: o formato JSON é de fácil leitura tanto por humanos quanto por máquinas.

Compatibilidade: é amplamente suportado pela maioria das linguagens de programação, sem necessidade de bibliotecas externas complexas.

Estrutura: permite representar objetos e listas de forma clara, refletindo diretamente as classes do sistema (ex.: lista de produtos, pedidos e clientes).

Flexibilidade: facilita a adição de novos campos sem quebrar a estrutura existente.

Os arquivos `.txt` foram descartados por não oferecerem estrutura adequada para dados complexos. O `.xml`, apesar de robusto, foi considerado menos prático para o escopo do projeto.

4.2 Divisão Inicial de Tarefas

Samuel: responsável pela apresentação do sistema, explicando o que é a loja virtual, para quem serve e quais funcionalidades ela terá.

Rafael: responsável por listar as classes principais do sistema, como `Cliente`, `Produto`, `Carrinho`, `Pedido`, `Pagamento` e `Administrador`, explicando o papel de cada uma.

Gustavo Ribeiro: responsável por mostrar o relacionamento entre as classes, podendo utilizar um diagrama UML simples ou explicar como elas se conectam entre si.

Gustavo: responsável por explicar os 4 pilares da POO aplicados ao sistema: abstração, encapsulamento, herança e polimorfismo.

Christian: responsável pelas decisões iniciais do projeto, como a escolha do formato de armazenamento, e pela organização da divisão de tarefas entre os integrantes.

Vitor: responsável por criar o repositório no GitHub, fazer o README com o nome do projeto, integrantes e descrição.

5 - Arquitetura do Sistema

O projeto é organizado em três camadas:

- Models/ — classes de domínio (Usuario, Produto, Carrinho, Pedido, Pagamento, Endereco)
- Repositorio.cs — responsável por carregar e salvar todos os dados em arquivos JSON
- Program.cs — menu principal, login e fluxo de interação com o usuário

6- Padrões de Projeto

O projeto não utiliza Design Patterns formais, mas adota três práticas reconhecidas:

- **Template Method (informal):** as classes abstratas Usuario e Pagamento definem a estrutura dos métodos, e cada subclasse preenche o comportamento específico.
- **Repository (simplificado):** a classe Repositorio centraliza toda leitura e gravação em JSON, isolando o restante do sistema do mecanismo de persistência.
- **Single Responsibility:** cada classe tem uma única responsabilidade — Produto cuida dos dados do produto, Carrinho cuida do carrinho, Pedido cuida do pedido.

7- Repositório do GitHub

[VitorCesar11/POO-Store](https://github.com/VitorCesar11/POO-Store)